

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA

Pedro Henrique Gimenez (matrícula 23102766)

Tom Pereira Hunt (matrícula 23102770)

Gustavo Rodrigues Alves D'Angelo (matrícula 23102761)

**Encurtador de URLs Distribuído: uma arquitetura cliente,
interceptador e servidor com Cache-Aside e Circuit Breaker**

INE5418 – Computação Distribuída
Trabalho 1, Semestre 2026/1

Sumário

1	Arquitetura e Decisões de Implementação	3
1.1	Linguagens	3
1.2	Protocolo	3
1.3	API REST	3
1.4	Cache-Aside	3
1.5	Circuit Breaker	3
1.6	Concorrência e configuração	4
1.7	Execução.	4
2	Resultados de Execução	4
2.1	Cache	5
2.2	Circuit Breaker	6
2.3	Parametrizações alternativas	7
3	Conclusão e Limitações	9

1 Arquitetura e Decisões de Implementação

O sistema tem três processos independentes: clientes, interceptador (*proxy* TCP) e servidor REST. Clientes e interceptador se comunicam por *sockets* TCP carregando mensagens JSON delimitadas por nova linha. O interceptador e servidor se comunicam por HTTP/REST. Do ponto de vista do cliente, o interceptador é o servidor.

1.1 Linguagens

Servidor REST e interceptador em Python (`http.server + threading + requests`). Bibliotecas cliente em Python e Node.js (esta usa só a *stdlib net*, sem `npm install`).

1.2 Protocolo

JSON por linha sobre TCP. Cada mensagem é um objeto JSON em uma única linha terminada por `\n`.

1.3 API REST

Endpoints `POST /urls`, `GET /urls/{codigo}`, `DELETE /urls/{codigo}`, `GET /urls`. Armazenamento em `dict` Python protegido por `threading.Lock`. O contador `acessos` é incrementado só no `GET /urls/{codigo}`.

1.4 Cache-Aside

Implementado em `interceptor/cache.py` em `OrderedDict`: LRU (`move_to_end + popitem(last=False)`) e TTL (cada entrada guarda `time.monotonic()`; no `get`, se `now - ts > ttl`, trata como *miss*). *Thread-safe* via `RLock`. Métricas: `hits`, `misses`, `evictions`, `expirations`, `invalidations`, acessíveis via ação `stats`. O cache é populado só em `resolve()` após *miss*. A invalidação no `remove` é incondicional, mesmo se o servidor responde 404.

1.5 Circuit Breaker

Escolhemos o Circuit Breaker e não *Throttling/Retry/Bulkhead* porque é complementar a cache e trata casos de muitas falhas em requests ao servidor. Implementado

em `interceptor/circuit_breaker.py` com estados `CLOSED`, `OPEN` e `HALF_OPEN`. `CLOSED` conta falhas consecutivas. Ao atingir o limite, vai pra `OPEN`. `OPEN` rejeita de imediato qualquer request. Depois do `reset_timeout`, a próxima chamada é tratada como teste em `HALF_OPEN`. Se der sucesso, volta a `CLOSED`, caso falhe, devolve a `OPEN`. Critério de falha: `requests.RequestException (ConnectionError, Timeout)` e HTTP 5xx. Erros 4xx não contam pois são erros vindos do cliente.

1.6 Concorrência e configuração

Cada conexão TCP gera uma `threading.Thread daemon`; `LRUCache` e `CircuitBreaker` são *thread-safe* via `RLock`. Não usamos `asyncio` porque `requests` é síncrono e o volume esperado é baixo. O arquivo `config.txt` (formato `KEY=VALUE`) define `SERVER_HOST/PORT`, `INTERCEPTOR_HOST/PORT`, `CACHE_MAX_SIZE`, `CACHE_TTL_SECONDS`, `CB_FAILURE_THRESHOLD`, `CB_RESET_TIMEOUT_SECONDS` e `CB_REQUEST_TIMEOUT`, que também poderiam ter sido configuradas como variáveis de ambiente.

1.7 Execução.

```
pip install -r requirements.txt
./start_all.sh                # sobe servidor REST e interceptador
python3 examples/example_python.py    # demo cliente Python
node      examples/example_javascript.js  # demo cliente Node.js
python3 examples/demo_cache.py         # demo cache (HIT/MISS/TTL)
python3 examples/demo_circuit_breaker.py # demo CB
./stop_all.sh
```

Logs em `logs/server.log` e `logs/interceptor.log`; PIDs em `logs/*.pid`.

2 Resultados de Execução

Todas as saídas foram coletadas em `localhost` com a configuração padrão (`CACHE_MAX_SIZE=64`, `CACHE_TTL_SECONDS=60`, `CB_FAILURE_THRESHOLD=3`, `CB_RESET_TIMEOUT_SECONDS` exceto onde indicado. O servidor e o interceptador foram reinicializados antes de cada execução para zerar contadores e cache.

2.1 Cache

Saída literal de `python3 examples/example_python.py`:

```
== encurta 3 URLs ==
encurta('https://en.wikipedia.org/wiki/Distributed_computing') -> rc=0 codigo=Hnagzh
    curta=http://127.0.0.1:5000/r/Hnagzh
encurta('https://docs.python.org/3/library/socket.html') -> rc=0 codigo=1wGH70 curta
    =http://127.0.0.1:5000/r/1wGH70
encurta('https://martinfowler.com/bliki/CircuitBreaker.html') -> rc=0 codigo=UzDKgu
    curta=http://127.0.0.1:5000/r/UzDKgu

== resolve 3x do mesmo codigo (1a MISS, demais HIT) ==
resolve(Hnagzh) tentativa 1 -> rc=0 url=https://en.wikipedia.org/wiki/
    Distributed_computing
resolve(Hnagzh) tentativa 2 -> rc=0 url=https://en.wikipedia.org/wiki/
    Distributed_computing
resolve(Hnagzh) tentativa 3 -> rc=0 url=https://en.wikipedia.org/wiki/
    Distributed_computing

== listagem ==
- {'codigo': 'Hnagzh', 'url_original': 'https://en.wikipedia.org/wiki/
    Distributed_computing', 'acessos': 1}
- {'codigo': '1wGH70', 'url_original': 'https://docs.python.org/3/library/socket.
    html', 'acessos': 0}
- {'codigo': 'UzDKgu', 'url_original': 'https://martinfowler.com/bliki/
    CircuitBreaker.html', 'acessos': 0}

== remove primeiro ==
remove(Hnagzh) -> rc=0

== resolve do que foi removido (deve dar 404) ==
resolve(Hnagzh) -> rc=404 (esperado 404)

== stats ==
cache:  {'hits': 2, 'misses': 2, 'evictions': 0, 'expirations': 0, 'invalidations':
    1} size=0/64 ttl=60.0s
circuit: {'state': 'CLOSED', 'failure_count': 0, 'threshold': 3, 'reset_timeout':
    8.0, 'total_calls': 7, 'successes': 7, 'failures': 0, 'short_circuits': 0, '
    recent_state_changes': []}
```

O contador `acessos=1` após três `resolve` do mesmo código é a prova do mecanismo da cache: das três invocações, apenas a primeira chegou ao servidor (o contador é incrementado no `server.py`). As outras duas viraram `hit` na Cache do interceptador, fechando em `hits=2`, `misses=2` (dois miss: o primeiro `resolve` e o `resolve` pós-remoção). `invalidations=1` mostra que o `remove` limpou a entrada antes do `resolve` final, que por isso devolveu 404.

2.2 Circuit Breaker

Saída literal de `python3 examples/demo_circuit_breaker.py` (servidor é forçado a 503 via `/_debug/fail`; três falhas abrem o circuito):

```
== estado inicial ==
circuit: {'state': 'CLOSED', 'failure_count': 0, 'threshold': 3, 'reset_timeout':
  8.0, 'total_calls': 0, 'successes': 0, 'failures': 0, 'short_circuits': 0, '
  recent_state_changes': []}

== servidor saudavel -> encurta normalmente ==
rc=0 codigo=W57sCV

== forçando fail_mode no servidor ==
POST /_debug/fail -> 200 {"fail_mode": true}

== 5 requisicoes -> circuito deve abrir apos 3 falhas ==
tentativa #1: rc=503 CB.state=CLOSED fails=1 short_circuits=0
tentativa #2: rc=503 CB.state=CLOSED fails=2 short_circuits=0
tentativa #3: rc=503 CB.state=OPEN fails=3 short_circuits=0
tentativa #4: rc=503 CB.state=OPEN fails=3 short_circuits=1
tentativa #5: rc=503 CB.state=OPEN fails=3 short_circuits=2

== chamada com circuito OPEN (fail-fast esperado) ==
rc=503 tempo=0.1ms

== aguardando 9s para HALF_OPEN ==

== proxima chamada: HALF_OPEN -> sucesso -> CLOSED ==
rc=0 codigo=ec67p0
CB.state=CLOSED successes=2
```

Nas tentativas #4 e #5 o `rc=503` não veio do servidor: o circuito já estava `OPEN` e o interceptador rejeitou antes de qualquer `HTTP` sair (registrado em `short_circuits`). Tempo de resposta na casa do décimo de ms em vez de segundos de *timeout* `TCP` é a *fail fast* na prática. Após `reset_timeout=8s`, a próxima requisição transita para `HALF_OPEN`; o sucesso fecha o circuito de volta para `CLOSED`.

2.3 Parametrizações alternativas

A configuração padrão exibe apenas `hits/misses/invalidations`; para exercitar `evictions` e `expirations` usamos duas configurações específicas. *Config A* (`CACHE_MAX_SIZE=2`, `CACHE_TTL_SECONDS=0`): três `encurta` seguidos de três `resolve` forçam *eviction* `LRU` na terceira inserção.

```
== Config A: CACHE_MAX_SIZE=2, CACHE_TTL_SECONDS=0 ==
encurta(3): codigos=['xb19BH', 'h01Qjb', 'yGAvvF']
resolve(xb19BH) -> rc=0  cache_size=1  cache={'hits': 0, 'misses': 1, 'evictions':
    0, 'expirations': 0, 'invalidations': 0}
resolve(h01Qjb) -> rc=0  cache_size=2  cache={'hits': 0, 'misses': 2, 'evictions':
    0, 'expirations': 0, 'invalidations': 0}
resolve(yGAvvF) -> rc=0  cache_size=2  cache={'hits': 0, 'misses': 3, 'evictions':
    1, 'expirations': 0, 'invalidations': 0}
```

Config B (`CACHE_MAX_SIZE=64`, `CACHE_TTL_SECONDS=2`): um `resolve` popula o cache, `sleep(3)` e o segundo `resolve` encontra a entrada expirada.

```
== Config B: CACHE_MAX_SIZE=64, CACHE_TTL_SECONDS=2 ==
encurta -> codigo=KUkool
resolve(KUkool) #1 -> rc=0  cache_size=1  cache={'hits': 0, 'misses': 1, 'evictions':
    0, 'expirations': 0, 'invalidations': 0}
sleep(3)...
resolve(KUkool) #2 -> rc=0  cache_size=1  cache={'hits': 0, 'misses': 2, 'evictions':
    0, 'expirations': 1, 'invalidations': 0}
```

Em A, o terceiro `resolve` insere a chave nova e o `LRU` descarta a mais antiga (`evictions=1`, `cache_size=2`). Em B, o segundo `resolve` detecta `TTL` vencido, contabiliza `expirations=1` e faz fallback para o servidor.

Heterogeneidade. O cliente `Node.js` consome o mesmo interceptador implementando o protocolo `JSON` por linha do zero, sem nenhuma dependência além da *stdlib*

(net). Saída literal de node `examples/example_javascript.js`:

```
-- ping
ping -> true

-- encurta duas URLs
encurta(https://nodejs.org/api/net.html) -> rc=0 codigo=bol3Dl curta=http
    ://127.0.0.1:5000/r/bol3Dl
encurta(https://learn.microsoft.com/azure/architecture/patterns/cache-aside) -> rc=0
    codigo=LPBwI4 curta=http://127.0.0.1:5000/r/LPBwI4

-- resolve 3x - observe o source
resolve(bol3Dl) #1 -> rc=0 source=server url=https://nodejs.org/api/net.html
resolve(bol3Dl) #2 -> rc=0 source=cache url=https://nodejs.org/api/net.html
resolve(bol3Dl) #3 -> rc=0 source=cache url=https://nodejs.org/api/net.html

-- remove + tenta resolver de novo
removeUrl(bol3Dl) -> rc=0
resolve(bol3Dl) apos remocao -> rc=404 (esperado 404)

-- listagem atual
- {"codigo":"LPBwI4","url_original":"https://learn.microsoft.com/azure/architecture/
  patterns/cache-aside","acessos":0}

-- stats
cache:  { hits: 2, misses: 2, evictions: 0, expirations: 0, invalidations: 1 } size
    =0/64 ttl=60s
circuit: {
  state: 'CLOSED',
  failure_count: 0,
  threshold: 3,
  reset_timeout: 8,
  total_calls: 6,
  successes: 6,
  failures: 0,
  short_circuits: 0,
  recent_state_changes: []
}
```

A primeira `resolve` devolve `source=server` e as duas seguintes devolvem `source=cache`,

deixando inequívoca a origem da resposta. Os contadores finais (`hits=2`, `misses=2`, `invalidations=1`) batem com o cenário Python, confirmando que o cache vive no interceptor e é compartilhado entre clientes independentemente da linguagem.

3 Conclusão e Limitações

A implementação da Cache no Proxy não é extensa, mas exigiu um cuidado quanto a política de invalidação através da invocação `remove`, não retornando entradas que expiraram. Preferiu-se pela estratégia conservadora de invalidar a Cache incondicionalmente, mesmo com o servidor respondendo 404, pois evita erros de coerência. Certamente, há o trade quanto a eficiência de perder algo na Cache que ainda está no Servidor e ter que retomar invocações.

O Circuit Breaker (CB), também curto em implementação, já apresenta mudanças significativas no comportamento: o cliente recebe o erro em $\sim 1\text{ms}$ em vez de travar dezenas de segundos esperando *timeout* TCP. Assim, podemos concluir que o CB, além de evitar travamento no Servidor, também evita que o Cliente aguarde desnecessariamente.